模型(Model)

一些经典模型

SPP-net(空间金字塔池化网络)

> 核心思想：通过空间金字塔池化层（SPP层），从任意尺寸的特征图中生成固定长度的特征向量。

> SPPNet 的核心贡献在于：通过空间金字塔池化层，使 CNN 能处理任意大小的输入图像，并在目标检测中实现卷积特征共享，为 Fast R-CNN 奠定了基础。

背景：在传统 CNN（如 AlexNet）中：

- 网络要求输入图像的尺寸固定（例如 224×224）。
- 不同尺寸的图片必须裁剪或缩放，会造成图像信息损失。

流程：

1. **卷积层**：从整张图片提取特征（可以输入任意大小图像）
2. **SPP层**：将最后一层卷积特征图做多尺度池化（1×1、2×2、4×4）
3. **全连接层（FC）**：输入固定长度的向量
4. **分类/检测层**：执行具体任务（如分类、检测）

Fast R-CNN

提出背景：

模型	问题
R-CNN (2014)	每个候选区域都要单独通过 CNN，计算量极大（数千次前向传播）
SPPNet (2014)	共享卷积特征，但训练不是端到端的（SPP层之后的全连接层单独训练）

于是 Fast R-CNN 解决了这两个问题：

✅ **1. 一次卷积提取整张图的特征（共享卷积）**
✅ **2. 实现端到端反向传播训练**

流程：

- 整张输入图片输入 CNN（例如 VGG16），得到一个 **feature map**（特征图）
- 通过 **Selective Search** 或其他方法产生约 2000 个候选区域（Region of Interest, ROI）。
- 候选区域结合到特征图中
- 引入ROI池化层，把每个候选区域也就是特征区域得到一个固定大小的特征图
- 分类与回归：判断ROI属于哪个类别、精确调整这个ROI边界框的位置使得更加贴合物体

关键优势：多任务损失 (Multi-task Loss)：

Fast R-CNN 的另一大进步是简化了训练过程。R-CNN 需要分三步训练：先微调 CNN，再训练 SVM 分类器，最后再训练边界框回归器。

Fast R-CNN 使用一个**多任务损失函数（Multi-task Loss）**，将**分类损失**（用于分类分支）和**回归损失**（用于回归分支）合并在一起。这使得整个网络（除了区域提议）可以**在一个阶段中进行端到端的统一训练**，大大简化了流程，也提升了精度。

Faster R-CNN

> Faster R-CNN的提出，使得用于目标检测的基于区域提议的 CNN 架构真正可以以端到端的方式进行训练

提出背景：

Fast R-CNN的第一步是**在模型外部**使用 Selective Search (SS) 算法来生成候选区域（RoI）。虽然 Fast R-CNN 内部的检测速度很快，但这个 Selective Search 算法本身非常耗时，它在 CPU 上运行，成为了整个系统的性能瓶颈。

Faster R-CNN 的**革命性贡献**就是：**用一个神经网络（RPN）来代替缓慢的 Selective Search 算法**，从而将“区域提议”这一步也集成到了 GPU 上的深度学习模型中，实现了**第一个真正意义上的、端到端的（End-to-End）目标检测框架。**

Faster R-CNN 的核心创新：区域提议网络 (RPN)

区域提议网络（Region Proposal Network, RPN）：

RPN的位置：和 Fast R-CNN 的检测头一样，**直接连接在主干 CNN（如 VGG-16）输出的全局特征图（Feature Map）上。**

RPN的工作原理：

- RPN 会在特征图上进行滑动窗口扫描。
- 在特征图的**每一个**位置，RPN 都会设想出 k个预设好不同尺寸和长宽比的“**锚框**”（Anchor Boxes）。例如，它可能会在同一位置同时考虑 3 种尺寸 * 3 种长宽比 = 9 个锚框。
- 对于锚框，**前景/背景分类（Objectness Score）：** 判断这个锚框里包含的是“前景（物体）”还是“背景”，这是一个二分类问题。
- **边界框回归（Bounding Box Regression）：** 如果它认为是“前景”，它会输出 4 个值，用于微调这个锚框的位置，使其更贴近真实的物体。

RPN 的优势：

- **共享计算：** RPN 直接使用主干 CNN 提取好的特征图，几乎是“零成本”地附加了一个提议功能。
- **速度极快：** 整个过程都在 GPU 上通过卷积实现，远快于在 CPU 上运行的 Selective Search。
- **质量更高：** RPN 是可以被训练的，它会学习如何生成高质量的提议，而不是像 SS 那样使用固定的算法。

流程与Fast R-CNN差不多，也就是在全局特征图之后有一个RPN，替换了Selective Search (SS) 算法

R-FCN

背景：为了**解决 Faster R-CNN 遗留的一个效率问题**，并实现一个**完全共享计算**的检测器。

Faster R-CNN 的**效率瓶颈**：RPN 提出了几百个 RoI，然后对每个Roi都执行了Roi Pooling,然后对每个Roi独立的流过几个全连接层，最后输出分类和回归结果。这里的瓶颈在于，虽然主干的卷积层是共享的，但 RoI Pooling 之后的**全连接层（FC layers）是不共享的。**如果有 300 个 RoI，这几层全连接层就要被重复计算 300 次，这在计算上仍然是昂贵的。

R-FCN 的目标就是：**把这几层昂贵的 FC 层也变成卷积层，并让所有 RoI 共享它们！**

核心矛盾：位置敏感性

为什么 Faster R-CNN 不直接把 FC 层换成卷积层呢？

这里有一个核心矛盾：

- **图像分类**任务（如 VGG, ResNet）追求**平移不变性 (Translation-Invariance)**。即一张猫的图片，无论猫在左上角还是右下角，都应该被识别为“猫”。卷积层和池化层天然具有这种特性。
- **目标检测**任务（R-CNN系列）追求**位置敏感性 (Position-Sensitivity)**。即模型必须知道 RoI 里的物体“在什么位置”。一个 RoI 框住“猫头”和框住“猫尾”是完全不同的。

Faster R-CNN 的解决方案是：前面用“平移不变”的卷积层提取深度特征，然后在 RoI Pooling 之后，用“位置敏感”的全连接层（FC layers）来做最后的分类和定位。

R-FCN 的挑战是：如何在*不使用 FC 层***（即全程保持全卷积）的前提下，解决这个“位置敏感性”的问题？

R-FCN的核心创新：

1. 位置敏感得分图 (Position-Sensitive Score Maps)

R-FCN 不再让主干网络最后只输出一个“特征图”。相反，它在最后加了一个卷积层，专门用来生成**一组**特殊的“位置敏感得分图”。

1. 位置敏感 RoI 池化 (Position-Sensitive RoI Pooling)

当 RPN 提出了一个 RoI 后，R-FCN 不使用普通的 RoI Pooling，而是使用 **PS-RoI Pooling**，它不再是对整个 RoI 区域进行池化。而是**只从对应的位置敏感得分图中**进行池化。再对整个区域中的得分进行平均，如果Roi只是框住了猫的一部分，那么平均分会拉的很低从而实现了**位置敏感性**

R-FCN 的最大贡献： 它将 Faster R-CNN 中 RoI 之后的昂贵的全连接层（FC layers）全部移除，替换为了在 RoI 之前计算的“位置敏感得分图”。这使得**几乎所有的计算都在整张图上共享**，**每个 RoI 只需要进行非常轻量级（几乎为零成本）的池化和平均操作**，从而大大提升了检测速度。

FPN(特征金字塔网络)

> 需要注意的是，FPN是一个**通用的“颈部”（Neck）模块**而**不是一个独立的目标检测器**（像 Faster R-CNN 那样）专门用来插入到各种检测器（如 Faster R-CNN, Mask R-CNN, RetinaNet 等）中，以**解决多尺度目标检测**的问题。

FPN解决的问题：多尺度检测

在传统的目标检测器在处理巨大的尺度差异会遇到困难如：使用单一的**深层特征**图，空间分辨率很低，小物体的所有细节都丢失了很难被检测，使用单一的**浅层特征图**语义信息很弱，模型模型“不知道”它看到的是什么，导致误检率高。**使用图像金字塔 (Image Pyramid)**（R-CNN 的做法）：将输入图像缩放成多种尺寸（比如 600px, 800px, 1000px...），每种尺寸都送入网络检测一次。效果好但是速度极慢，计算量和内存开销成倍增加，无法用于实时检测。

FPN 的思想是：**我们能否在不增加巨大计算量的前提下，构造一个*特征*金字塔，使得金字塔的每一层都既有高分辨率，又有丰富的语义信息？**

自顶向下路径 (Top-down Pathway)：

- 这是 FPN 的关键。它从最深的特征图 C_5 开始，将其丰富的语义信息“传递”回浅层。
- **步骤 1：** 将 C_5 通过一个 1×1 卷积（用于统一通道数），得到 P_5。P_5 是金字塔的顶层。
- **步骤 2：** 将 P_5 进行**上采样**（Upsampling，通常是 2 倍），使其空间尺寸与 C_4 相同。

- **步骤 3：** 将 C_4 也通过一个 1×1 卷积（称为**横向连接, Lateral Connection**），使其通道数与 P_5 匹配。
- **步骤 4：** 将上采样后的 P_5 和 1×1 卷积后的 C_4 进行**逐元素相加 (Element-wise Addition)**。
- **步骤 5：** 将相加的结果再通过一个 3×3 卷积（用于消除上采样带来的混叠效应），最终得到 P_4 。

这组特征图的特点是：**每一层都具有丰富的语义信息**，但它们的空间分辨率不同（P_2 最大，P_5 最小）。

FPN 如何用于检测：

当 FPN 被集成到 Faster R-CNN 中时，RPN（区域提议网络）和 Fast R-CNN（检测头）**不再只连接在最后一层特征图上。**

- **RPN** 会在 $\{P_2, P_3, P_4, P_5, P_6\}$ **每一层**上独立运行，并根据不同层的特点使用不同大小的锚框（Anchor）。
- **检测头**（RoI Pooling + 分类回归）也会根据 RoI 的大小，从金字塔的不同层中抽取特征。
  - **小的 RoI** 会从高分辨率的层（如 P_2, P_3）中抽取特征。
  - **大的 RoI** 会从低分辨率的层（如 P_4, P_5）中抽取特征。

Maskrcnn

实例分割 (Instance Segmentation)：不仅知道“这是狗”，还知道“这是*哪一只*狗”以及“它*具体长什么样*”。

两个阶段完成任务：

一、RPN找目标,图片上扫一次，找出所有可能有物体的区域，**产出** 一堆“候选框”(Regions of Interest, RoIs)

二、精细分析，对每一个候选框分类、微调边框、抠图(把框内的物体精确轮廓画出来，生成一个Mask，也就是**掩码生成 (Mask Generation)**)

Mask R-CNN 的秘诀 (RoIAlign)：

- 它发明了一种新技术，叫做 **RoIAlign**。
- **比喻：** 它不再给专家一个模糊的截图，而是给了专家一个**“高清、无损、完美对齐”**的放大镜。
- 专家可以通过这个“高清放大镜”看到原始图像上最精确的像素信息，所以他描绘出来的轮廓（Mask）就**极其精确**，不会有错位和模糊。

YOLO

因其速度快多引用在实时目标检测

划分网格 (Grid)： YOLO 会把输入的图像划分成一个 S*S 的网格

网格负责制： 如果一个物体的*中心点*落在了某个网格（Grid Cell）内，那么**这个网格**就全权负责预测这个物体。

一次性预测所有：YOLO 的网络非常巧妙，它在一次前向传播中，会让每一个网格都同时预测出几样东西：

- **边界框 (BBox)：** 预测 N 个边界框的坐标（x, y, w, h）
- **置信度 (Confidence)：** 这个框里*有物体*的概率有多大？（这个框和真实物体的重合度有多好？）

- **类别概率 (Class Probability)：** 如果有物体，它分别是猫、狗、人...的概率各是多少？

“撒网捕鱼”： 你可以想象 YOLO 像是在图像上撒了一张大网（S * S 的网格），每个网眼（Grid Cell）都同时扔出了几个“小渔网”（预测框）。

NMS 清理： 这张大网撒下去，肯定会有很多“小渔网”都网住了同一条鱼（同一个物体）。这时，YOLO 就必须调用 NMS 这个“指挥官”来清理现场，把多余的网都扔掉，只保留最好的那个。

SSD

DCNN

PAN

PAN（Path Aggregation Network）最早用于 YOLOv4，主要解决一个问题：

> **高层语义信息**（能判断“是什么”）在网络后层；
> **底层细节信息**（能精确定位“在哪里”）在网络前层。
>
> PAN 的目的，就是让语义与细节信息双向流动、充分融合。

简单理解：

- **FPN（Feature Pyramid Network）** 是自上而下的信息流，把高层语义传到低层；
- **PAN** 在此基础上增加了自下而上的路径，把定位信息传回高层。

与基于区域提议的模型相比，基于回归的模型(yolo,ssd)通常可以实时处理，但代价是精度下降。此外，通过引入其他技巧（PVANET），例如BN、残差连接，基于区域提议的模型可以修改为实时系统。

显着目标检测

总体而言，由于 CNN 主要提供局部区域的显着信息，因此大多数基于 CNN 的方法需要借助**超像素分割**对沿区域边界的视觉显着性进行建模。同时，多尺度深度CNN特征的提取对于测量局部显着性具有重要意义。最后，有必要加强不同 CNN 层之间的局部连接，并利用局部和全局上下文中的补充信息。

人脸检测

深度密集人脸检测器（DDFD）：它能够检测各种方向的人脸，而不需要姿势/地标注释

可以通过以下策略来**提高性能**：设计新颖的优化损失、修改通用检测管道、构建有意义的网络级联、采用尺度感知检测和学习多任务共享 CNN 特征。

行人检测

简而言之，基于CNN的方法可以为行人识别和定位提供更准确的候选框和多级语义信息。同时，手工制作的特征是互补的，可以与CNN结合以达到更好的效果。通过仔细设计框架和分类器、提取多尺度和基于部分的语义信息以及从其他相关任务（例如分割）中搜索补充信息，可以实现对现有 CNN 方法的改进。

展望未来方向和任务

小物体检测：为了提高部分遮挡下小物体的定位精度，需要从以下几个方面修改网络架构。

多任务联合优化与多模态信息融合

规模适应：需要训练尺度不变、多尺度或尺度自适应检测器

空间相关性和上下文建模

随着大规模图像和视频数据的出现，减轻了人工的负担，实现了实时的物体检测。可以考虑以下三个方面：

级联网络

无监督学习和弱监督学习

网络优化

扩展2D目标检测的典型方法，以适应3D目标检测和视频目标检测，以满足自动驾驶、智能交通和智能监控的需求：

3D 目标检测

视频对象检测：不同框架的时间信息在理解不同对象的行为方面发挥着重要作用。然而，准确性会受到视频中退化的对象外观（例如运动模糊和视频散焦）的影响，并且网络通常不是端到端训练的。为此，应考虑时空小管[230]、光流[199]和LSTM[107]从根本上对连续帧之间的对象关联进行建模。

模型验证

yolo验证

评价两个框相近程度的算法：IoU(intersection over Union)(iou就是两个框的交集面积除以并集面积)

在预测之前要设置正确的**IoU阈值**，大于这个阈值才算正确的预测(通常这个IoU设置为0.5)

IoU阈值:预测结果和某个真实结果类型相同并且IoU超过了0.5，那么这个预测结果才能算是正确的

为每种类型分别统计TP(true positive)(正确的预测结果), FP(false positive)(错误的预测结果), FN(false negative)(假负例，没找到的真实结果)

可以发现(TP + FP)就是**预测结果总数**,这里也可以理解为把所有预测结果分为了TP和FP

(TP + FN)就是**真实结果总数**，就是把所有真实结果分类为TP和FN

P(precision,精准率)　R(recall,召回率)　F1(F1分数)　三个指标：

P = 正确预测除以预测总数

R=正确预测除以真实总数

F1 = (2*P *R)%(P + R)

预测过程分为三个步骤：1.**预处理**(图片处理)，2.模型**推理**，3.**后处理**(去除掉大部分多余的预测结果，并把图缩放回原来的大小)

推理后处理：

处理掉多余的框的三种方法，后处理三联：

- 使用conf(confidence,概率)阈值筛选掉低概率的框，conf=0.25(排除置信度低于0.25的框)(**去弱**)
- 用**NMS(Non-Maximum Suppression, 非极大值抑制)**过滤掉重复的预测结果, iou=0.7(以IoU>0.7为标准，排除重复框)(**去重**)
- 限制预测结果总数, max_det=300(限制预测总数不超过300)(**限量**)

PR曲线(召回率为横坐标): 根据P曲线和R曲线可以得到PR曲线

PR曲线下面的面积是一个0-1的小数，这个面积称为**AP**(average precision,平均精度)

当P曲线和R曲线都非常优秀的时候，PR曲线也会非常优秀，AP也会越来越接近1

mAP(mean average precision,平均平均精度),就是把所有类型的AP平均一下

可以给每个mAP都增加一个后缀，暗示这个mAP是基于多大的IoU阈值，IoU阈值越高，对应mAP越低(因为要求更加严格了)

两个常用的指标：

mAP50(IoU阈值为0.5)

mAP50-95

IoU阈值从0.50-0.95，每过0.05计算一次mAP，把所有mAP平均一下就可以得到**mAP50-95**(平均平均平均精度)这个指标

从这里知道了验证时获取的**4个主要指标**：

- P
- R
- mAP50
- mAP50-95

训练流程梳理：训练集喂给模型->模型更新一次参数->用模型验证一下验证集看看效果->得到这4个指标->4个指标做一次加权平均(每个指标跟**权重**(默认情况4个指标的权重分别是0.0，0.0，0.1，0.9)相乘再把所有结果相加),得到一个**fitness(适应度)**(fitness体现了模型对验证集的适应程度)

每训练一轮会自动把本轮的fitness和目前为止最好的fitness对比，如果本轮的最好，那么本轮就是最优秀的模型，最优秀的模型保存，训练结果里面有一个best.pt文件就是用来保存最优秀的模型，每一轮会把当前模型保存为last.pt

---

DETR(DEtection TRansformer)

DETR的核心是“**一对一**” + **Transformer**，从而**彻底消灭了 NMS**。

核心思想：

DETR（一对一）： 指挥官（模型）手里有 100 个“令牌”（Object Queries）。他对士兵们说：“张三（令牌1），你必须且只许负责盯住敌人A；李四（令牌2），你必须且只许负责盯住敌人B... 王五（令牌99），你负责‘背景’...”

- **结果：** 实战时，每个令牌（士兵）只输出一个*唯一*的预测结果。张三报告敌人A的位置，李四报告敌人B的位置。**根本没有重复的框**，所以**不需要 NMS**。

DETR由三部分组成：骨干、核心、头部

核心 (Core) - Transformer：

- 这是 DETR 的“心脏”，它也分为两部分：
- **编码器 (Encoder)：** 负责“理解”整个图像。它接收 CNN 提取的特征图，并利用“自注意力”机制来分析图像中不同部分之间的*全局关系*（比如“天空”和“建筑”的边界，“人”和“自行车”的依赖关系）。
- **解码器 (Decoder)：** 负责“提问”并“回答”。
  - **提问 (Object Queries)：** 这是最巧妙的地方。DETR 不是在图像上“撒网”，而是初始化一堆（比如 100 个）固定的“**对象查询 (Object Queries)**”。你可以把这 100 个 Query 理解为 100 个“空的提问槽”或“等待分配任务的士兵”。
  - **回答：** 解码器让这 100 个 Query 与编码器“理解”后的图像信息进行交互。最终，每个 Query 都会“聚焦”到图上的一个特定物体（或背景）上，并输出它的预测结果。

头部 (Head) - FFN：

- 一个非常简单的前馈神经网络（FFN）。
- 它接收 Transformer 解码器输出的 100 个结果，并为**每一个**结果预测两件事：
  1. **类别：** 这是一个“人”、“车”、还是“背景 (no object)”？
  2. **边界框 (BBox)：** 它的位置 (x, y, w, h) 在哪里？

匈牙利损失 (Hungarian Loss)

工作原理：

1. 在训练的每一步，模型都会输出 100 个预测（每个 Query 一个）。
2. 我们有（比如）5 个真实物体（Ground Truth）。

3. **匈牙利算法**会介入，它会像一个“月老”一样，在 100 个预测和 5 个真实物体之间，计算出一个**成本最低的“一对一”配对方案。**
4. 比如，算法发现：[Query 7] 与 [真实物体A] 最像，[Query 53] 与 [真实物体B] 最像...
5. 配对完成后，模型就开始学习：[Query 7] 的目标就是无限接近 [真实物体A]，[Query 53] 的目标就是无限接近 [真实物体B]...
6. 至于其他没有被配对上的 95 个 Query 呢？它们的目标就是预测为“**背景 (no object)**”。

结果： 经过长时间训练，每个 Query 都学会了自己专门负责去寻找一个物体（或背景），模型从根本上学会了**不产生重复预测。**‘

DETR的缺点：

收敛慢： 需要非常长的训练时间才能学会这种“一对一”的分配。

对小物体检测困难： 原始 DETR 在这方面表现不佳。

YOLO10论文

背景： YOLO的检测流程由两部分组成：模型前处理和NMS后处理。然而，它们仍然存在缺陷，导致精度-延迟边界不理想。

yolo模型在预测的时候会有许许多多的锚框定位到了同一个目标，所以需要用NMS进行去重，但是使用NMS去重的时候，会降低推理速度，因为NMS本身是一个*额外*的计算步骤。模型跑完（神经网络前向传播）之后，你还得再跑一遍 NMS 算法来“清理”结果。如果一张图上有几千个预测框，NMS 也会很慢。使得阻止了端到端(神经网络输出就是最终答案，不需要后处理部分，是一个理想状态)的部署

论文的工作：

使用双标签分配和一致的匹配度量为无 NMS YOLO 提出一致的双分配策略来解决后处理中的冗余预测问题。它使模型在训练过程中享受丰富而和谐的监督，同时在推理过程中无需使用 NMS，从而获得高效率的竞争性能。其次，我们通过对 YOLO 中的各个组件进行全面检查，提出了模型架构的整体效率-准确性驱动的模型设计策略。为了提高效率，我们提出了轻量级分类头、空间通道解耦下采样和排序引导块设计，以减少明显的计算冗余并实现更高效的架构。为了提高准确性，探索了大核卷积并提出了有效的部分自注意力模块来增强模型能力，在低成本下挖掘性能改进的潜力。

双标签分配(鱼与熊掌兼得)：

传统yolo(一对多),训练效果好，但是必须用NMS去重

DETP(一对一),训练时用“匈牙利匹配”，推理时**不需要 NMS**（端到端），但训练更难，收敛慢（监督较弱）。

双标签分配就是这两个的优点的结合：

在原来的yolo头中增加了另一个一对一的头，保留了与原始一对多分支相同的结构并采用相同的优化目标，但利用一对一匹配来获取标签分配。训练过程中，两个头部和模型共同优化，在推理过程中，丢弃一对多头并利用一对一头进行预测

一对一就像种子选手，一对多就像陪练，训练的时候一起训练，种子选手得到了陪练团的魔鬼训练，在推理的时候，陪练团下场

一对多会产生多个预测，模型用“一致性匹配指标”（Consistent Match Metric）给所有这些预测打分，得分最高的框作为一对一头唯一的学习目标

Relation DETR

---

yoloWorld

模型架构：由 YOLO 检测器、文本编码器和可重参数化视觉语言路径聚合网络（RepVL-PAN）组成。给定输入文本，YOLO-World 中的文本编码器将文本编码为文本嵌入。 YOLO 检测器中的图像编码器从输入图像中提取多尺度特征。然后，我们利用 RepVL-PAN 通过利用图像特征和文本嵌入之间的跨模态融合来增强文本和图像表示。

- YOLO Detector：基于YOLO8开发，包含作为图像编码器的 Darknet 主干网 、用于多尺度特征金字塔的路径聚合网络（PAN）以及用于边界框回归和对象嵌入的头。
- Text Encoder：采用 CLIP 预训练的文本编码器来提取相应的文本嵌入 ，与纯文本语言编码器相比，CLIP 文本编码器提供了更好的视觉语义功能，用于将视觉对象与文本连接起来
- Text Contrastive Head：继之前的工作之后，我们采用具有两个 3×3 卷积的解耦头来回归边界框和对象嵌入
- Training with Online Vocabulary：在训练过程中，为每个包含 4 个图像的马赛克样本构建一个在线词汇表 T。具体来说，我们对马赛克图像中涉及的所有正名词进行采样，并从相应的数据集中随机采样一些负名词。每个马赛克样本的词汇最多包含M个名词，M默认设置为80。
- Inference with Offline Vocabulary：在推理阶段，提出了一种带有离线词汇的提示然后检测策略，以提高效率。用户可以定义一系列自定义提示，其中可能包括标题或类别。然后，我们利用文本编码器对这些提示进行编码并获得离线词汇嵌入。离线词汇可以避免对每个输入进行计算，并提供根据需要调整词汇的灵活性。

RepVL-PAN：RepVL-PAN（Representation Vision-Language PAN）

两个关键模块：

T-CSPLayer（Text-guided CSP Layer）

I-Pooling Attention（Image Pooling Attention）

整个 RepVL-PAN 是一个**双向信息交换系统**：

1. 文本 → 图像（通过 T-CSPLayer）
   让图像特征具有语言语义。
2. 图像 → 文本（通过 I-Pooling Attention）
   让文本嵌入对视觉内容有上下文感知。

经过这两个方向的交互后，输出的多尺度特征（P3、P4、P5）同时具备：

- 局部视觉细节；

- 全局语义理解；
- 跨模态对应能力。

最终，这些特征被送入检测头（Box Head + Text Contrastive Head）进行目标框预测与语义匹配。

yoloworld---旨在实现高效的开放词汇目标检测，遵循标准YOLO架构，利用预先训练的 CLIP文本编码器对输入文本进行编码，进一步提出可重新参数化的视觉语言路径聚合网络（RepVL-PAN）来连接文本特征和图像特征，以获得更好的视觉语义表示。在推理过程中，可以删除文本编码器，并将文本嵌入重新参数化为 RepVLPAN 的权重，以实现高效部署

用户的提示进行编码以构建离线词汇表，并且词汇表随着不同的需求而变化。然后，高效的检测器可以动态推断离线词汇，而无需重新编码提示。对于实际应用，一旦我们训练了检测器，即YOLO-World，我们就可以对提示或类别进行预编码以构建离线词汇表，然后将其无缝集成到检测器中。

yoloworkd的主要贡献：

- 我们引入了YOLO-World，这是一种尖端的开放词汇对象检测器，对于现实世界的应用具有高效率。
- 提出了一个可重新参数化的视觉语言PAN来连接视觉和语言特征，并为YOLO-World提出了一个开放词汇区域文本对比预训练方案。
- 提出的 YOLO-World 在大规模数据集上进行预训练，展示了强大的零样本性能，并在 LVIS 上以 52.0 FPS 实现了 35.4 AP。预训练的 YOLO-World 可以轻松适应下游任务，例如开放词汇实例分割和引用对象检测。

从语言编码器（例如 BERT ）中提取词汇知识来解决开放词汇检测

但有缺点：（1）繁重的计算负担和（2）边缘设备的复杂部署。

开放词汇对象检测（OVD）：旨在检测预定义类别之外的对象。

预训练：

预训练 YOLO-World 的训练方案

- Learning from Region-Text Contrastive Loss：
- Pseudo Labeling with Image-Text Data：标记方法包含三个步骤：（1）提取名词短语：我们首先利用名词提取算法从文本中提取名词短语；（2）伪标记：我们采用预训练的开放词汇检测器，例如GLIP [24]，为每个图像的给定名词短语生成伪框，从而提供粗略的区域文本对。（3）过滤：我们采用预训练的CLIP [39]来评估图像-文本对和区域-文本对的相关性，并过滤低相关性的伪注释和图像。我们通过结合非极大值抑制（NMS）等方法进一步过滤冗余边界框